cfRNA ME/CFS Classifier and Explorer — Project Plan

cfRNA ME/CFS Classifier & Explorer — Project Plan
1. Dataset snapshot
2. Data reality check before building anything
3. Why this dataset is a good practice problem
4. Suggested pipeline
5. Splitting PyTorch vs. TensorFlow practice
6. Research questions tailored to this dataset
7. FastAPI layer
8. Phased roadmap
9. Caveats specific to this dataset

cfRNA ME/CFS Classifier & Explorer — Project Plan

Dataset: GSE293840 — Circulating cell-free RNA (cfRNA) signatures for ME/CFS (Gardella et al., *PNAS*, 2025)

1. Dataset snapshot

This is the Cornell (De Vlaminck / Hanson / Grimson labs) cell-free RNA study published in *PNAS* in August 2025. Circulating cell-free RNA — RNA fragments released into blood during cell turnover — was profiled by RNA sequencing in plasma from 93 individuals with ME/CFS and 75 unaffected sedentary controls, with samples collected at Ithaca College, Weill Cornell Medicine, and the ID Med clinic in Los Angeles, generating about 40.8 million reads per sample on average. That's a notably larger, better-sequenced cohort than most public ME/CFS omics datasets.

The published benchmark to calibrate against: a generalized linear model with LASSO regression trained on condition-specific signatures achieved a **test-set AUC of 0.81** and an **accuracy of 77%**. The investigators found more than 700 differentially expressed transcripts distinguishing cases from controls, and the diagnostic model was ultimately built on 21 genes. On the biology side, immune cfRNA deconvolution revealed differences in platelet-derived cfRNA between cases and controls, plus elevated plasmacytoid dendritic cell, monocyte, and T-cell-derived cfRNA in ME/CFS, with network analysis pointing to cytokine signaling and T-cell exhaustion signatures. A notable negative finding worth knowing: they did **not** find a significant difference in overall circulating viral burden between groups.

2. Data reality check before building anything

The raw counts matrix is deposited as a GEO supplementary file (`GSE293840_raw_counts_all.csv.gz` — genes × samples). Two things to sort out immediately:

- **Case/control labels aren't in the counts file.** Pull sample metadata (condition, age, sex, collection site, possibly sequencing batch) from the GEO series matrix / SOFT file with `GEOparse.get_GEO("GSE293840")`, and join on sample ID (GSM accession).
- **This is raw counts, not normalized expression.** Before any modeling, apply library-size normalization (CPM or median-of-ratios) and a log transform (`log1p`). Skipping this is the single most common beginner mistake with RNA-seq count data — raw counts are dominated by sequencing depth differences between samples, not biology.

3. Why this dataset is a good practice problem

168 samples is still “small-N” by general ML standards, but with roughly 20,000+ genes as features it’s a classic small-N-large-P problem — exactly what teaches dimensionality reduction, regularization, and honest cross-validation instead of throwing a big network at the data and hoping. And because there’s a real published AUC (0.81) to calibrate against, there’s an actual sanity check: a model reporting AUC 0.99 signals a leakage bug, not a win.

4. Suggested pipeline

```
Raw counts (GEO) → normalize (CPM + log1p) → filter low-expression genes
  → feature selection (top variable genes / differential expression)
  → train/val/test split (stratified, site-aware)
  → model → FastAPI service
```

5. Splitting PyTorch vs. TensorFlow practice

Rather than building the same MLP twice, give each framework a distinct job:

- **TensorFlow/Keras — unsupervised stage:** a small autoencoder that compresses the ~2,000 top-variable-gene input down to a 16–32 dimensional latent space. Use the embeddings for (a) visualization (UMAP/t-SNE) and (b) unsupervised clustering to check whether ME/CFS cases separate into subgroups — a good, real research question given the disease’s known heterogeneity.
- **PyTorch — supervised stage:** the actual classifier, trained either on the top variable genes directly or on the autoencoder’s latent features. This is the model wrapped in FastAPI. Use k-fold cross-validation (not a single split) and compare against a plain sklearn logistic regression/LASSO baseline — replicating the *type* of model the original paper used is a good sanity check before trusting a fancier one.

6. Research questions tailored to this dataset

1. **Replication:** Using your own feature selection and a regularized linear model, can you get in the neighborhood of the paper’s AUC 0.81 / 21-gene signature? (You won’t have their exact gene list without digging into supplementary tables, so this is about approach, not exact reproduction.)
2. **Does deep learning help here at all?** With ~168 samples and thousands of genes, does a PyTorch MLP or the autoencoder+classifier combo meaningfully beat a LASSO baseline — or does it just overfit? A genuinely open, honest question with this data.
3. **Subgroup structure:** Does the TensorFlow autoencoder’s latent space show clustering within ME/CFS cases (by sex, site, or symptom severity if available) rather than one uniform group?
4. **Site/batch effects:** With three collection sites, does site act as a confounder? A model that secretly learns “which clinic collected this sample” instead of “ME/CFS vs. control” is a real risk with multi-site data — worth explicitly testing (e.g., can a classifier predict site from the control samples alone, above chance?).
5. **Feature interpretability check:** Pull SHAP values or integrated gradients from your best model and see whether the top contributing genes land in the same territory the paper reports (immune function, cytokine signaling, T-cell exhaustion) — a real plausibility check, not just a metric.

7. FastAPI layer

- `POST /predict` — takes a gene expression vector (post-normalization), returns predicted class, probability, and top contributing genes (SHAP). The disclaimer field references the actual published benchmark (AUC 0.81) so the API is honest about where it stands relative to the literature.
- `GET /model/metadata` — surfaces cross-validated AUC alongside the paper’s reported 0.81, so the comparison is visible directly rather than a bare number.

- `GET /dataset/summary` — sample counts by group, by site, gene count after filtering.

8. Phased roadmap

1. **Data wrangling (few days):** GEOparse for metadata, pandas for the counts matrix, CPM+log1p normalization, join labels.
2. **Baseline (few days):** sklearn LASSO logistic regression with stratified k-fold — the benchmark to beat, and the first real comparison point against the published 0.81 AUC.
3. **TensorFlow autoencoder (about a week):** latent embedding + clustering/visualization.
4. **PyTorch classifier (about a week):** MLP on top-variable-genes or autoencoder embeddings, k-fold CV, SHAP.
5. **FastAPI (3–5 days):** wrap it, as designed above.
6. **Stretch:** site-effect diagnostic, subgroup analysis writeup, Dockerize + deploy.

9. Caveats specific to this dataset

- cfRNA composition is sensitive to preanalytical handling (time-to-processing, freeze-thaw) — differences found could reflect collection-site logistics as much as biology, which is exactly why checking for a site confound (research question 4) matters here more than in a single-site study.
- 93 vs. 75 is good for this field but still means wide confidence intervals — report cross-validated ranges, not a point estimate.
- The original team already did the immune-deconvolution analysis with a proprietary cell-type atlas from their own lab (Grimson lab) — exact reproduction of “which immune cells” isn’t possible without it, so treat those questions as directional, not a replication target.